OPENBRIDGE

OpenBridge — Acelerador de Integraciones Open Source

Documentación de la IA · estado al 11 de noviembre de 2026

Equipo S10-26-equipo 29 · Simulación laboral No Country (octubre–noviembre 2026) · Demo final: 13 de noviembre de 2026

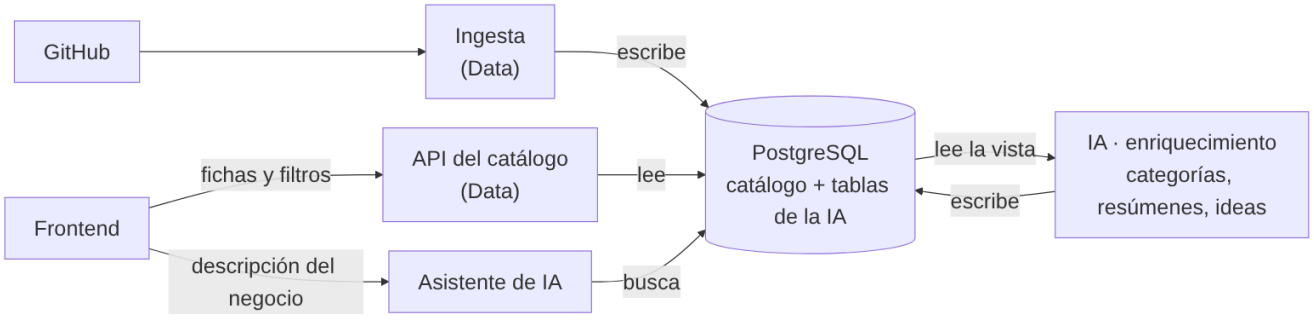
David Vergara	Project Manager
Jeison Pinzón	Data Engineer
Andrés Iglesias	AI Engineer
Fabián Martínez	UX/UI Designer · Frontend

Resumen

Lo que hoy le cuesta a un arquitecto entre 15 y 20 horas de búsqueda, OpenBridge lo empieza con una propuesta justificada en segundos.

El problema. El cliente es un estudio de desarrollo y consultoría que hace más de 30 proyectos al año. En cada uno, sus arquitectos dedican entre 15 y 20 horas a buscar herramientas open source a mano (cifra del cliente). El resultado: esfuerzo repetido, librerías abandonadas que se adoptan sin saberlo y alternativas gratuitas que se pasan por alto.

Qué es OpenBridge. Un catálogo web de herramientas open source con ideas de integración, que se actualiza desde GitHub, y un asistente con IA: el usuario describe su negocio y recibe un stack recomendado y justificado. La ingesta y la API del catálogo son de Data Engineering; el frontend, de UX/UI. **Este documento cubre la IA.**



Qué gana el cliente

- **Rapidez:** describe su negocio con sus palabras y recibe en segundos un stack de herramientas open source, una por cada necesidad.
- **Confianza:** todo lo que se recomienda existe en el catálogo, y cada recomendación explica, con las palabras del cliente, para qué le sirve y cómo se conecta con las demás.
- **Siempre al día:** el catálogo y los datos de la IA se actualizan solos cada día.

La IA en tres frases. El asistente propone una herramienta del catálogo por cada necesidad del negocio y, si el catálogo no tiene lo que hace falta, lo dice en lugar de inventar. En una prueba final con pedidos de

negocio escritos sin ver el catálogo, **el usuario recibió una herramienta correcta en el 69 % de las necesidades**; cuando la herramienta correcta llega a los candidatos, el asistente casi siempre la elige, y lo que falta mejorar está en la búsqueda (sección 3). Cada decisión se tomó con una medición detrás (sección 4), los datos de la IA se actualizan solos cada día, y lo que todavía no funciona se dice con datos, junto con el próximo paso (sección 5).

1. Qué hace la IA

La IA tiene cinco piezas, pensadas para que el cliente reciba una recomendación en la que pueda confiar: la búsqueda semántica, el asistente, el enriquecimiento del catálogo, las ideas de integración y la actualización diaria de sus datos. Todas siguen la misma regla: **el asistente solo recomienda herramientas que están en el catálogo, y si ninguna sirve, lo dice.** Cuánto se cumple cada parte está medido en la sección 3.

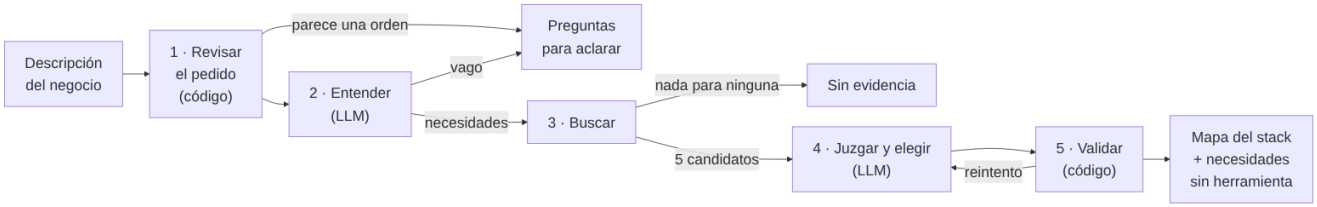
1.1 Búsqueda semántica

Recibe una necesidad en lenguaje natural, en español o en inglés (por ejemplo, "cobrar suscripciones mensuales"), y devuelve las herramientas del catálogo más parecidas por significado, no por palabras exactas.

- **Cómo funciona:** cada repositorio se convierte en un vector (*embedding*) a partir de su nombre, sus categorías, su descripción, sus temas de GitHub y el resumen y los casos de uso que genera la IA (sección 1.3). La consulta se convierte igual y se comparan.
- **Modelo:** EmbeddingGemma-300m, en CPU, sin GPU ni servicios de pago. Los vectores se guardan en la base de datos: al arrancar no se vuelven a calcular.
- **Primero, la categoría de la necesidad:** la búsqueda trae primero las herramientas cuya categoría principal coincide con la de la necesidad y, si no alcanzan, completa con la búsqueda abierta.
- **Umbral de "sin evidencia":** si nada en el catálogo se parece lo suficiente, la necesidad queda sin candidatos en lugar de traer la herramienta menos mala.

1.2 Asistente

El usuario describe su negocio y recibe un mapa de su stack: una herramienta del catálogo por cada necesidad, con su justificación, y las necesidades que el catálogo no cubre, dichas como tales.



1. **Revisar el pedido:** si el texto intenta darle órdenes al asistente (por ejemplo, "ignora tus reglas"), responde con preguntas para aclarar, sin llamar al modelo de lenguaje.
2. **Entender:** un modelo de lenguaje separa el pedido en necesidades, de una lista cerrada de 18 categorías. Si el pedido es demasiado vago, el asistente pregunta en lugar de suponer.
3. **Buscar:** la búsqueda semántica trae hasta 5 candidatos por necesidad.
4. **Juzgar y elegir:** el modelo juzga si cada candidato hace lo que se pidió antes de nombrarlo, y elige **una sola herramienta entre sus candidatos, o ninguna**. Justifica la elección con las palabras del usuario. Las necesidades sin herramienta pasan a "Limitaciones" de la respuesta.

5. **Validar:** el código comprueba la respuesta contra el contrato acordado con el frontend y rechaza herramientas que no estaban entre los candidatos, enlaces, imágenes o copias de las instrucciones internas. Si algo no cumple, pide la respuesta otra vez, una sola vez.

El asistente es un servicio propio, al que llama el frontend. Cada pedido tiene un tiempo máximo de 15 segundos. El modelo principal es Gemini 3.5 Flash-Lite; si falla por algo temporal (límite de uso, caída o demora), la misma llamada pasa sin esperar a un modelo de respaldo en Groq. Si no hay respuesta válida, el frontend recibe un código de error y elige qué mensaje mostrar. Cambiar o agregar un proveedor de modelo es una entrada en un registro, sin tocar el resto del asistente. El costo de cada pedido no depende del tamaño del catálogo: el modelo ve como máximo 5 candidatos por necesidad (unos 3.500 tokens de entrada).

1.3 Enriquecimiento del catálogo

Para cada repositorio, un modelo de lenguaje lee su README y produce:

- **De 1 a 3 categorías** de la lista de 18 (la primera es la principal), para los filtros del catálogo y para la búsqueda.
- **Un resumen en español** de hasta 250 caracteres, el que muestra la ficha de cada herramienta.
- **Casos de uso**, cada uno con una cita del README que el código comprueba. Si el README no los respalda, el campo queda vacío.

1.4 Ideas de integración

Una idea de integración explica cómo conectar una herramienta con la aplicación del usuario: tipo de conexión (API REST, Docker, SDK o webhook), evento que la dispara, stack compatible, dependencias, dificultad, un fragmento de código solo si está copiado tal cual de la documentación, y el enlace que la respalda.

Ejemplo, generado por la IA a partir del README de Novu:

> **Enviar notificaciones multicanal con Novu** · API REST · dificultad media
>
> Envía notificaciones a través de múltiples canales como Inbox, Email, SMS, Push y Chat usando una única API HTTP unificada. Se dispara con un evento importante en la aplicación o una acción del usuario que requiere aviso.

Reglas:

- De 1 a 3 ideas por herramienta, cada una con una cita del README que el código comprueba. **Si ningún tipo de conexión encaja con evidencia, la herramienta queda sin ideas:** preferimos una ficha sin ideas a una con una idea falsa.
- Las ideas generadas se publican como "**sin revisar**", y una idea que revisó una persona nunca se sobrescribe.

1.5 Actualización diaria

Los datos de la IA se mantienen solos. Todos los días a las 7:17 (hora de Bogotá), una tarea programada en GitHub Actions compara el catálogo con lo publicado y regenera solo los repositorios cuya descripción, temas o README cambiaron. Un cambio de estrellas o de fechas no gasta llamadas al modelo.

- **Publica sin revisión humana, con un freno:** si de una vez cambió más del 40 % del catálogo, o harían falta más de 100 llamadas al modelo, se detiene sin publicar nada.

- **Avisa:** si se detiene o falla, abre un aviso (*issue*) en el repositorio con el informe, y lo cierra solo cuando una corrida termina bien.
- **Sin reiniciar:** el asistente revisa cada 10 minutos si hay datos nuevos y los carga sin dejar de atender; un pedido en curso termina con los datos anteriores.

Cada dato que publica la IA guarda una huella de lo que leyó y de la versión de las reglas con que se generó. Por eso la actualización sabe qué cambió y qué no.

2. Por qué esta arquitectura

Recomendar herramientas a partir de la descripción de un negocio admite varias soluciones. Elegimos la que combina lo que un modelo de lenguaje hace bien —entender un pedido escrito con palabras de negocio y juzgar si una herramienta sirve— con lo que el código puede garantizar: que todo salga del catálogo, con una estructura fija y validada.

Enfoque	Qué hace bien	Por qué no alcanza
Un chatbot con un modelo de lenguaje, sin catálogo	Entiende cualquier pedido y escribe bien	Recomienda de memoria: puede nombrar herramientas abandonadas o que no existen, y nada lo comprueba
Un buscador por palabras clave y filtros	Rápido y predecible	No entiende el pedido: "cobrar las cuotas" no coincide con un repositorio que dice *billing*
Traer documentos y dejar que el modelo responda libre (RAG simple)	Responde con información del catálogo	La respuesta es texto libre: no garantiza una herramienta por necesidad, ni conexiones con estructura, ni rechaza lo que no está en el catálogo
OpenBridge: pasos con responsabilidades separadas	El modelo entiende y juzga; la búsqueda semántica encuentra; el código valida	Más piezas que mantener, y la calidad depende de la búsqueda (sección 3)

Es un razonamiento de diseño: no construimos ni medimos los otros tres enfoques. Lo respaldan dos cosas que sí observamos: sin una lista cerrada, un modelo inventó una categoría que no existía, y el juicio sobre cada candidato redujo las recomendaciones que no servían (sección 4).

2.1 El ciclo de una consulta

1. **El cliente escribe su pedido** en el frontend, que lo envía al servicio de la IA.
2. **El servicio ya tiene todo cargado:** el catálogo publicado en la base, sus vectores y el modelo de embeddings. Nada pesado se calcula por pedido.
3. **Revisar, entender, buscar, juzgar y elegir, validar** (sección 1.2), con una o dos llamadas al modelo de lenguaje, más un reintento si algo no cumple.
4. **La respuesta vuelve con una estructura fija** —herramientas, conexiones, supuestos y limitaciones— y el frontend la dibuja como un mapa del stack, con acceso a la ficha de cada herramienta.
5. **En segundo plano,** la actualización diaria publica los datos nuevos y el servicio los toma sin reiniciarse (sección 1.5).

Si el proveedor principal falla, la misma llamada pasa al respaldo sin esperar; si no hay una respuesta válida dentro de los 15 segundos, el frontend recibe un código de error y muestra un mensaje claro.

2.2 Por qué Gemini 3.5 Flash-Lite

Los criterios se fijaron antes de comparar: costo cero o casi para desarrollo y demo; por API, porque el servicio no tiene GPU; salida con un esquema JSON; español natural; todo el pedido en menos de 15 segundos; y elegir solo entre los candidatos.

- **Gemini 3.5 Flash-Lite (elegido):** sin errores del proveedor, cumplió las reglas al primer intento y respondió en unos 4 segundos de mediana.
- **gpt-oss-120b en Groq (respaldo):** el más rápido y también cumple al primer intento, pero su cuota por minuto alcanza para cubrir una caída, no para atender todo el tráfico.
- **Claude Haiku (evaluado, no por defecto):** eligió y explicó algo mejor, pero en 2 de cada 3 pedidos su primer intento rompía las reglas del diseño; con el reintento tardaba unos 10 segundos de mediana, y es de pago.
- **Descartados antes:** un modelo local (Qwen 3.5 4B con Ollama) tardaba entre 12 y 16 segundos solo en generar; Gemma 4 26B entraba en bucle al escribir el JSON; Gemini 3.1 Flash-Lite tuvo errores del servidor durante horas.

La comparación fue de 5 pedidos × 3 corridas (3-nov): suficiente para descartar por errores y tiempos, no para dar porcentajes de acierto, que por eso no se publican. Cambiar de modelo es una variable de configuración. Al decidir, Gemini se usaba en su nivel gratuito; hoy el proyecto usa un crédito prepago (sección 5).

3. Cómo sabemos que funciona

Cómo se midió. Las cifras de esta sección vienen de una **prueba final**: casos guardados en el repositorio antes de medir, que no se usaron para tomar ninguna decisión, y que se corrieron una sola vez con la versión ya elegida. Los pedidos de negocio los escribió otro modelo de lenguaje sin ver el catálogo, se filtraron con reglas fijas y no se editaron. Las respuestas correctas se pusieron después. Durante el desarrollo usamos otros conjuntos de casos. Sus cifras salen más altas (por ejemplo, 92 % frente a 73 % en búsqueda) porque se usaron para decidir o se escribieron conociendo el catálogo. Por eso no se publican como rendimiento.

Versión medida: `main` en `ba94ef4` · catálogo de 127 repositorios (8-nov) · búsqueda medida el 8-nov y asistente el 9-nov.

Qué se midió	Resultado	Sobre qué
El usuario recibe una herramienta correcta	**69 %** (123 de 177)	16 pedidos de negocio, 59 necesidades con herramienta en el catálogo, 3 corridas
↳ El asistente detecta la necesidad	93 % (165 de 177)	
↳ La herramienta correcta llega a los 5 candidatos	79 % (131 de 165)	necesidades detectadas
↳ El asistente la elige cuando llega	94 % (123 de 131)	
Búsqueda: la herramienta correcta entre las 3 primeras	73 % (41 de 56)	56 consultas con respuesta

Qué se midió	Resultado	Sobre qué
Tiempo por pedido	5,0 s (mediana)	48 pedidos
Defensas contra manipulación: ataques que resisten	7 de 10	casos guardados antes de programar las defensas · 9-nov
Pedidos legítimos bloqueados por error	0 de 6	ídem
Respuestas sin tildes	31 de 610 (5,1 %)	respuestas de Gemini · 9-nov
Datos de la IA en el catálogo	127/127 con categoría y resumen; 123 con casos de uso; 58 con ideas de integración (78)	archivos del repositorio, 10-nov

Qué dice el embudo. Cuando la herramienta correcta llega a los candidatos, el asistente casi siempre la elige. La pérdida está antes, en la búsqueda. Por tipo de necesidad, todo acierta en inicio de sesión (18/18), almacenamiento (24/24), búsqueda (12/12) y observabilidad (6/6). Los puntos débiles son base de datos (1/21) y reportes sobre datos propios (8/21).

Después de esta medición se integraron dos mejoras dirigidas al punto más débil: el 10-nov, la búsqueda da prioridad a la categoría de cada necesidad, y el 11-nov, el paso de entender recibe ejemplos parecidos al pedido para detectar necesidades dichas con palabras de negocio ("llevar un historial"). Las dos mejoraron en desarrollo, sobre todo en bases de datos, y sin empeorar el resto; esas cifras no se publican porque no pasaron por una prueba final.

Lo que no se puede afirmar todavía. En la prueba final solo 2 de las 61 necesidades no tenían herramienta en el catálogo, muy pocas para dar una cifra. En desarrollo, cuando el catálogo no tiene la herramienta, el usuario lo ve como limitación en el 50 % y el 38 % de los casos (dos conjuntos). Ese es hoy el punto más débil (ver sección 5).

Cobertura no es calidad. Que un repositorio tenga resumen o ideas no significa que estén bien. La calidad se revisó en muestras, no en todo el catálogo.

4. Decisiones de ingeniería

Las decisiones de la IA que más pesan, con la alternativa que se descartó, por qué y qué las respalda. Las evidencias con fecha son de cuando se tomó la decisión; el rendimiento actual está en la sección 3.

Decisión	Alternativa descartada	Por qué	Evidencia
Lo que el código puede comprobar no se le deja al modelo	Que el modelo de lenguaje arme toda la respuesta	El modelo decide solo lo que requiere criterio (qué necesita el negocio, qué herramienta sirve); la estructura, las categorías y los candidatos válidos los controla el código.	Sin lista cerrada, un modelo inventó una categoría ("authentication"); con ella, otro escribió "IA" en vez de `ai`, y la validación lo detectó y el reintento lo corrigió.

Decisión	Alternativa descartada	Por qué	Evidencia
Elegir solo entre candidatos, con un juicio antes de nombrar	Bloquear repositorios con listas, o subir el umbral de la búsqueda	Una regla general escala con el catálogo; una lista de repositorios no. El umbral solo frena pedidos sin nada parecido en el catálogo: el freno real es el juicio sobre cada candidato.	Al decidirlo (7-nov, catálogo de 100), el asistente se abstuvo en 64–73 % de las necesidades sin herramienta, frente a 42–54 % antes (10 casos, 3 corridas), sin dejar de recomendar las herramientas que sí sirven.
Gemini 3.5 Flash-Lite como modelo principal, Groq como respaldo	Claude Haiku (de pago) o un modelo local	Sin errores, cumple las reglas al primer intento y responde muy por debajo del tiempo máximo. Groq es el más rápido, pero su cuota alcanza para cubrir una caída, no para reemplazarlo.	Comparación de 5 pedidos × 3 corridas (3-nov); detalle en la sección 2.2.
Embeddings con EmbeddingGemma-300m	Qwen3-Embedding-0.6B	En acierto quedaron parejos; decidieron la velocidad, la memoria y que separa mejor los pedidos que el catálogo no cubre.	Muestra de 20 repositorios y 14 consultas (26-oct): 3 a 4 veces más rápido y ~1,2 GB frente a ~2,4 GB.
Buscar primero en la categoría de la necesidad, con respaldo abierto	Solo búsqueda abierta, o filtrar estrictamente por categoría	Una base de datos se parece en el texto a herramientas de almacenamiento de archivos o caché; la categoría las separa. El respaldo abierto evita perder una herramienta mal categorizada.	Medido en desarrollo con pedidos nuevos escritos sin ver el catálogo (10-nov): mejoró las bases de datos sin empeorar la elección ni la abstención.
Defensas contra manipulación en el código, por tipo de ataque	Pedirle al propio modelo que se defienda	Lo que se revisa antes de llamar al modelo no depende de que el modelo obedezca. Las reglas salen de tipos generales de ataque, no de los casos de prueba.	Casos de prueba guardados antes de programar las defensas, con el criterio fijado de antemano (resultado en la sección 3).
Actualización diaria automática, con freno	Revisión humana antes de publicar, o actualizar a mano	Un equipo chico no sostiene una revisión diaria; sin automatizar, los datos de la IA quedan viejos. El freno limita el daño de un cambio masivo.	Dos corridas manuales del proceso completo sin errores (10-nov). El riesgo que deja está en la sección 5.
Usuarios de base de datos con privilegio mínimo	Un único usuario con todos los permisos	Un error de la IA no puede dañar el catálogo de Data, y el lector de la API no puede escribir.	Verificado: el usuario de la IA escribe solo en sus tablas; no puede crear tablas ni leer la tabla base del catálogo.
Medir con una prueba final escrita sin ver el catálogo	Medir con los casos usados durante el desarrollo	Los casos con que se decide dan cifras optimistas; la prueba final mide como lo usaría un cliente.	Las cifras de desarrollo salieron más altas que las de la prueba final (sección 3).

5. Límites conocidos y hoja de ruta

Un sistema confiable es uno que sabe dónde falla. Esto es lo que medimos que todavía falta y el próximo paso de cada punto; dentro de cada grupo, de más a menos importante.

El asistente

- **A veces recomienda una herramienta parecida que no resuelve la necesidad**, cuando el catálogo no tiene la correcta: para chat entre personas propone un servicio de notificaciones, y para copias de seguridad automáticas, un almacenamiento de archivos. Es el punto más débil medido (sección 3). → *Próximo paso:* darle al juicio de cada candidato su resumen en español y pedirle una cita literal de por qué sirve; medirlo con casos nuevos.
- **No detecta algunas necesidades**, y entonces el usuario ni siquiera ve la limitación: necesidades "casi" (permisos por sucursal, flujo de aprobación, red de distribución de contenido) y, en la versión medida, registrar los datos del negocio ("llevar un historial", "controlar las reservas"), que el asistente trataba como parte de la aplicación y no como una base de datos. → *Próximo paso:* la mejora de detección integrada el 11-nov (sección 3) ataca el segundo caso; falta confirmarla con una prueba final nueva, con casos nuevos. Servir contenido "desde servidores cercanos" sigue sin detectarse. Más adelante: pedirle al usuario que confirme las necesidades dudosas.
- **Bases de datos y reportes sobre datos propios son los tipos de necesidad más débiles** (sección 3). Dos causas encontradas: la descripción de PostgreSQL en GitHub es pobre y queda abajo dentro de su categoría, y para "guardar los datos" el modelo a veces elige una librería de acceso a datos en vez de la base. → *Próximo paso:* la búsqueda por categoría (sección 4) ya mejoró esto en desarrollo; confirmarlo con una prueba final nueva. Después, buscar también con el pedido completo y no solo con cada necesidad.
- **La herramienta elegida cambia entre corridas:** en 14 de los 16 pedidos de la prueba final, el asistente no dio el mismo resultado las 3 veces, muchas veces alternando entre dos herramientas correctas.
- **Algunas respuestas llegan sin tildes** (sección 3), casi siempre la respuesta entera. → *Próximo paso:* si un texto largo no tiene ninguna tilde, pedirlo otra vez; medirlo con casos nuevos.
- **El asistente todavía no usa las ideas de integración** para explicar cómo se conectan las herramientas que recomienda. → *Próximo paso:* decidir si el modelo ve las ideas antes o después de elegir, y medirlo con casos nuevos.

Seguridad

- **Los ataques que no tienen forma de orden pasan:** un pedido cortés, una orden con las letras separadas y, sobre todo, una descripción de repositorio manipulada en tono cortés, que llega a las justificaciones en las 3 corridas. Ese texto entra por los datos del catálogo, que la actualización diaria publica sin revisión humana. → *Próximo paso (el más importante de la lista):* revisar la descripción y el README de cada repositorio nuevo o cambiado antes de publicar sus datos, con un detector medido con casos nuevos.
- **El modelo de respaldo no se probó contra ataques.** → *Próximo paso:* correr con él la misma prueba.

Datos que genera la IA

- **Ninguna persona revisó todo el catálogo:** la calidad del enriquecimiento y de las ideas se midió en muestras (16 repositorios para el enriquecimiento; 15 + 3 para las ideas). Hay categorías secundarias

dudosas y algunos resúmenes con traducciones literales. → *Próximo paso:* revisar una muestra con un criterio que escale, no repositorio por repositorio.

- **Más de la mitad de los repositorios no tiene ideas de integración.** La mayoría son herramientas para construir o trabajar en una aplicación, no para conectarse a ella; pero algunos son fallos conocidos: los frameworks son inconsistentes (Angular, Vue, Svelte y Flask quedan sin ideas, React no). → *Próximo paso:* ideas a partir del conocimiento general del modelo, marcadas con su origen, sin fragmentos de código y revisadas en una muestra.

Operación

- **Depende de un solo proveedor con crédito prepago.** Si se agota el crédito de Gemini, el respaldo de Groq tiene una cuota diaria chica, compartida con las evaluaciones. Además, en una corrida de desarrollo (10-nov), 8 de 75 pedidos se cortaron por superar el tiempo máximo de 15 segundos. → *Próximo paso:* sumar otros proveedores, con los criterios de elección fijados antes de comparar.
- **La actualización diaria publica sin revisión humana.** El freno corta los cambios masivos, pero no un cambio chico y malicioso (ver Seguridad).

Cómo se midió

- **Los conjuntos de prueba son chicos** (16 pedidos, 10 ataques): las cifras marcan una dirección, no son estadística firme.
- **Varios conjuntos de desarrollo los redactó un asistente de IA después de leer el catálogo**, y por eso son más fáciles que los pedidos de un usuario real. La prueba final no tiene ese problema, pero ya se usó una vez: la próxima mejora necesita casos nuevos (el procedimiento está escrito en el repositorio).
- **Las cifras y el umbral de la búsqueda valen para una versión del catálogo.** Al crecer, hay que volver a etiquetar los casos (hay un comando para eso) y recalibrar. Hoy el umbral depende de una sola consulta. → *Próximo paso:* una regla de umbral que no dependa de un solo caso.

Para crecer

- **La búsqueda corre en memoria.** Con miles de repositorios conviene moverla a la base de datos (pgvector).
- **No se hizo una prueba de carga** del asistente con varios usuarios a la vez.

6. Cómo probarlo

En vivo: openbridge.up.railway.app, disponible hasta el 28 de noviembre de 2026. Basta con describir un negocio en el asistente.

En tu máquina: desde la raíz del repositorio, con Python 3.12 y uv. Requisitos: una cuenta de Hugging Face con la licencia de EmbeddingGemma aceptada (el modelo pesa ~1,2 GB y corre en CPU) y una clave de Gemini en `.env` (los nombres están en `.env.example`). Sin acceso a la base de datos, el asistente usa la copia del catálogo que está en el repositorio.

```
uv venv --python 3.12
source .venv/Scripts/activate      # Git Bash en Windows (Linux / macOS: .venv/bin/activate)
uv pip install -r ai/requirements.txt

python -m pytest ai/tests -q       # tests de la IA: sin el modelo ni claves
python -m ai.eval.run_search_eval --golden catalog-test   # búsqueda con la prueba final (requiere el modelo)

AI_CATALOG_SOURCE=files python -m uvicorn ai.service.app:create_app --factory --port 8001
```

La cifra de la búsqueda puede no coincidir con la sección 3: el catálogo y la búsqueda cambiaron después de medir. Con el servicio arriba:

- `GET http://127.0.0.1:8001/health` dice cuántos repositorios cargó y de dónde (base de datos o archivos).
- `http://127.0.0.1:8001/docs` permite probar `POST /assistant` desde el navegador, con un pedido como "Un gimnasio quiere cobrar las cuotas mensuales y avisar a sus socios por correo".

Paso a paso, variables de entorno y el resto de los comandos: README de la IA · cifras vigentes y cómo se midieron: línea base de la evaluación.